Integration Testing, Q2 Report

March 23, 2026

1 System Overview

StreetRace Manager is a command-line system built as a set of interacting modules. The required modules implemented are Registration, Crew Management, Inventory, Race Management, Results, and Mission Planning. Two additional modules were added: Fuel Management and Reputation System.

1.1 Modules Included

- **Registration**: registers crew members and stores their name, role, and active status.
- **Crew Management**: updates roles and skill levels, and exposes available crew by role.
- **Inventory**: stores vehicles, spare parts, tools, and the shared cash balance.
- **Race Management**: creates races, registers participants, and changes race state.
- **Results**: records race outcomes, calculates prizes, and updates rankings.
- **Mission Planning**: creates missions, checks required roles, assigns crew, and awards mission rewards.
- **Fuel Management**: tracks fuel usage, checks race readiness, and charges refueling cost.
- **Reputation System**: tracks reputation points, tier progression, and unlocked content.

1.2 Additional Modules Chosen

Fuel Management was added because race planning becomes more realistic when the system must check whether a car can actually complete a race. It also creates a direct integration with Inventory through refueling costs.

Reputation System was added because race outcomes can be used for longer-term progression. This module links completed races with driver progression and unlockable content.

2 Call Graph

The columns drawn include:

- Registration
- Crew Management
- Inventory
- Race Management
- Results
- Mission Planning
- Fuel Management
- Reputation System

2.1 The function nodes are as:

- **Registration**: `register_member`, `get_member`, `member_exists`, `get_members_by_role`, `activate_member`, `deactivate_member`
- **Crew Management**: `assign_role`, `update_skill_level`, `get_skill_level`, `list_crew_by_r`
  `get_available_drivers`, `get_available_mechanics`
- **Inventory**: `add_vehicle`, `get_vehicle`, `damage_vehicle`, `repair_vehicle`, `add_cash`, `deduct_cash`
- **Race Management**: `create_race`, `register_participant`, `start_race`, `complete_race`, `get_race`, `get_race_participants`
- **Results**: `record_race_result`, `get_driver_ranking`, `get_top_drivers`, `update_cash_balanc`
- **Mission Planning**: `create_mission`, `check_role_availability`, `check_roles_availability`
  `assign_mission`, `complete_mission`
- **Fuel Management**: `initialize_vehicle_fuel`, `get_fuel_level`, `check_fuel_sufficiency`, `consume_fuel`, `refuel_vehicle`
- **Reputation System**: `add_reputation`, `get_member_reputation`, `gain_reputation_from_rac`
  `get_unlocked_content`, `_update_tier`

2.2 Cross-Module Arrows

- `CrewManagementModule.assign_role` → `RegistrationModule.member_exists`, `RegistrationM`
- `CrewManagementModule.get_available_drivers` → `RegistrationModule.get_members_by_r`
- `RaceManagementModule.register_participant` → `RegistrationModule.get_member`, `InventoryModule.get_vehicle`

- `ResultsModule.record_race_result` → `RegistrationModule.get_member`
- `ResultsModule.update_cash_balance_from_race` → `InventoryModule.add_cash`
- `MissionPlanningModule.check_role_availability` → `RegistrationModule.get_members_by`
- `MissionPlanningModule.assign_mission` → `RegistrationModule.member_exists`, `RegistrationModule.get_member`
- `MissionPlanningModule.complete_mission` → `InventoryModule.add_cash`
- `FuelManagementModule.refuel_vehicle` → `InventoryModule.get_vehicle`, `InventoryModule`
- `FuelManagementModule.get_fuel_level` → `InventoryModule.get_vehicle`
- `ReputationSystem.add_reputation` → `RegistrationModule.get_member`, `ReputationSystem.`
- `ReputationSystem.get_reputation_status` → `RegistrationModule.get_member`

2.3 Call Graph Image

(Due to the sheer size of the image incresing the size of the report, I have not attached the image to te report. It can be found in integrtion/diagrams. Please look at it there)

3 Integration Test Design

The modules were initially individually tested and verified. Then, integration tests were designed to jointly test them.

The integration tests were written in `pytest` and stored in `integration/tests/test_integration.` The goal was to verify module interaction and data flow, not only isolated unit behaviour. Each case below states the scenario, modules involved, expected result, actual result, and why the test was needed.

ID	Scenario / Why Needed	Modules	Expected Result	Actual Result	Notes
T1	Assign role before registration. Needed to enforce the first business rule.	Registration, Crew Management	Error is raised.	Pass	Unregistered IDs are rejected.
T2	Register member and then assign role. Needed to confirm valid cross-module flow.	Registration, Crew Management	Role changes successfully.	Pass	Basic positive path.
T3	Request skill for missing member. Needed to ensure shared validation is applied.	Registration, Crew Management	Error is raised.	Pass	Prevents silent invalid state.

ID	Scenario / Why Needed	Modules	Expected Result	Actual Result	Notes
T4	List drivers after deactivating one. Needed because only active crew should be available.	Registration, Crew Management	Inactive driver excluded.	Pass after fix	Found bug: inactive members were returned. Fixed in `list_crew_by_role`
T5	Enter a mechanic into a race. Needed to enforce the driver-only racing rule.	Registration, Inventory, Race Management	Error is raised.	Pass	Non-drivers cannot race.
T6	Register a valid driver and vehicle for a race. Needed for positive race entry path.	Registration, Inventory, Race Management	Participant is added.	Pass	Stored as driver-vehicle pair.
T7	Enter an inactive driver into a race. Needed to check status flow from registration to races.	Registration, Inventory, Race Management	Error is raised.	Pass	Inactive members are blocked.
T8	Try using a damaged vehicle in a race. Needed to verify vehicle state affects race logic.	Registration, Inventory, Race Management	Error is raised.	Pass	Damaged vehicle is unavailable.
T9	Repair a damaged vehicle after adding parts. Needed to test repair workflow.	Registration, Inventory	Repair only works after required parts exist.	Pass	Vehicle returns to good condition.
T10	Check mechanic availability for repair-related mission logic. Needed to verify role availability checks.	Registration, Mission Planning	Mechanic reported unavailable.	Pass	Shows mechanic requirement is enforced.
T11	Finish a race and transfer prize money to inventory. Needed to validate results to cash flow.	Registration, Inventory, Race Management, Results	Inventory cash increases by winner prize.	Pass	Prize distribution works.
T12	Create mission without required mechanic available. Needed to enforce mission role rule.	Mission Planning, Registration	Required role reported unavailable.	Pass	Mission should not start.

ID	Scenario / Why Needed	Modules	Expected Result	Actual Result	Notes
T13	Create and assign mission when required role exists. Needed for positive mission path.	Registration, Mission Planning	Mission becomes active.	Pass	Crew assignment stored correctly.
T14	Assign inactive member to mission. Needed to propagate crew status into missions.	Registration, Mission Planning	Error is raised.	Pass	Inactive crew blocked.
T15	Full race workflow from registration to rankings. Needed as an end-to-end integration check.	Registration, Inventory, Race Management, Results, Fuel Management	Race starts, fuel is consumed, rankings update.	Pass	Core workflow succeeds.
T16	Full mission workflow with reward payout. Needed as an end-to-end mission integration check.	Registration, Mission Planning, Inventory	Mission completes and reward is added to cash.	Pass	Cash balance updated correctly.
T17	Award reputation after race win. Needed to connect race outcomes with progression.	Registration, Inventory, Race Management, Reputation System	Reputation points increase.	Pass	Winner gains 50 points.
T18	Unlock content at higher reputation tier. Needed to verify tier-based integration.	Registration, Reputation System	Content becomes available at threshold.	Pass	Silver unlock content observed.
T19	Drain fuel and verify race readiness fails until refuel. Needed because fuel is an added planning constraint.	Inventory, Fuel Management	Fuel becomes insufficient, then sufficient after refuel.	Pass	Fuel gating works.
T20	Refuel a car and check cash balance decreases. Needed to verify shared cash state.	Inventory, Fuel Management	Cash decreases after refuel.	Pass	Refueling charges inventory.
T21	Request fuel for unknown vehicle ID. Needed to prevent operations on missing inventory items.	Inventory, Fuel Management	Error is raised.	Pass after fix	Found bug: fuel module accepted non-existent vehicles. Fixed with vehicle validation.

ID	Scenario / Why Needed	Modules	Expected Result	Actual Result	Notes
T22	Register member with invalid role. Needed as an input validation edge case.	Registration	Error is raised.	Pass	Invalid role blocked.
T23	Create race with invalid race type. Needed to validate race setup input.	Race Management	Error is raised.	Pass	Invalid type rejected.
T24	Start race with one participant only. Needed to enforce minimum race size.	Registration, Inventory, Race Management	Error is raised.	Pass	Minimum of two enforced.
T25	Assign skill level above allowed range. Needed to check boundary validation.	Registration, Crew Management	Error is raised.	Pass	Skills remain bounded.
T26	Deduct more cash than available. Needed to protect shared inventory state.	Inventory	Error is raised.	Pass	Negative cash prevented.
T27	Query reputation for unknown member. Needed to stop phantom reputation records.	Registration, Reputation System	Error is raised.	Pass after fix	Found bug: reputation was created for unknown IDs. Fixed with member validation.
T28	Run two races at the same time. Needed to confirm state isolation between race objects.	Registration, Inventory, Race Management	Both races keep correct participants.	Pass	No cross-race contamination.
T29	Build leaderboard from multiple completed races. Needed to test long-term results aggregation.	Registration, Inventory, Race Management, Results	Rankings generated in points order.	Pass	Each driver ends with at least one win.

4 Errors Found and Fixes Applied

- **Packaging issue in tests**: running `pytest integration/tests` from the repository root failed because Python could not find the modules in `integration/code`. This was fixed by adding `integration/tests/conftest.py` to place the code directory on the import path.
- **Inactive crew leak in role listings**: `CrewManagementModule.list_crew_by_role()`

claimed to return active crew, but previously returned inactive members too. The function was updated to filter on `is_active`.

- **Fuel operations on unknown vehicles**: the fuel module previously accepted IDs that did not exist in inventory. Validation was added so fuel queries and refueling only work for real vehicles.
- **Phantom reputation records**: the reputation module previously created data for unknown member IDs. Validation was added so reputation can only be queried or updated for registered members.

5 Actual Test Outcome

All integration tests passed after the fixes above.

- Command used: `python -m pytest integration/tests -q`
- Final result: `29 passed`